DBMS ASSIGNMENT

ASSIGNMENT 8

Basic SQL Queries and Aggregate Functions

Project: Library Loan Management System

Database: MySQL 8.0

Objective

Use WHERE, ORDER BY, GROUP BY, HAVING, aggregate functions, and safe UPDATE/transaction examples on the library dataset.

Note: outputs are expected results based on the included sample data. Run the SQL in your MySQL environment and insert your own screenshots if required by your instructor.

1. Introduction

The queries use LibraryLoanDB and the sample data from Assignment 7. Overdue loan results are evaluated as of 22 October 2026, so the date is explicit and reproducible.

2. Find Overdue Loans Using WHERE

```
SELECT LoanID, MemberID, CopyID, DueOn
FROM Loans
WHERE ReturnedOn IS NULL AND DueOn < '2026-10-22'
ORDER BY DueOn, LoanID;

Expected output:
LoanID | MemberID | CopyID | DueOn
9002 | 22 | 503 | 2026-09-03
9001 | 21 | 501 | 2026-09-21
9015 | 24 | 506 | 2026-10-19

Explanation: WHERE filters open loans whose due dates are earlier than the reference date. Returned
loans are excluded.
```

3. Sort Catalogue Using ORDER BY

```
SELECT BookID, Title, PublicationYear
FROM Books
WHERE Category = 'Computing'
ORDER BY PublicationYear DESC, BookID;

Expected output:
103 | Python Lab Notes | 2025
104 | Networks Made Simple | 2024

Explanation: the newest computing title is displayed first; BookID provides a stable tie-breaker.
```

4. Count Loans per Borrowing Member

```
SELECT MemberID, COUNT(*) AS LoanCount
FROM Loans
GROUP BY MemberID
ORDER BY LoanCount DESC, MemberID;

Expected output:
24 | 3
25 | 3
26 | 3
27 | 3
21 | 2
22 | 2
28 | 2
29 | 2

Explanation: GROUP BY creates one result row per member with at least one loan. Members with zero loans
are included in Assignment 9 using LEFT JOIN.
```

5. Frequent Borrowers Using HAVING

```
SELECT MemberID, COUNT(*) AS LoanCount
FROM Loans
GROUP BY MemberID
HAVING COUNT(*) >= 3
ORDER BY MemberID;

Expected output:
24 | 3
25 | 3
26 | 3
27 | 3

Explanation: HAVING filters groups after COUNT has been calculated.
```

6. Aggregate Functions on Returned Loans

```
SELECT COUNT(*) AS ReturnedLoans,
       SUM(DATEDIFF(ReturnedOn, BorrowedOn)) AS TotalDays,
       ROUND(AVG(DATEDIFF(ReturnedOn, BorrowedOn)), 2) AS AvgDays,
       MIN(DATEDIFF(ReturnedOn, BorrowedOn)) AS MinDays,
       MAX(DATEDIFF(ReturnedOn, BorrowedOn)) AS MaxDays
FROM Loans WHERE ReturnedOn IS NOT NULL;

Expected output:
ReturnedLoans | TotalDays | AvgDays | MinDays | MaxDays
13 | 173 | 13.31 | 9 | 18

Explanation: COUNT, SUM, AVG, MIN and MAX summarize completed borrowing durations. DATEDIFF counts
elapsed calendar days, not inclusive days.
```

7. Summarize Copy Condition

```
SELECT ConditionStatus, COUNT(*) AS Copies
FROM BookCopies
GROUP BY ConditionStatus
ORDER BY Copies DESC, ConditionStatus;

Expected output:
Good | 13
Damaged | 1
Lost | 1
Withdrawn | 1

Explanation: condition is not the same as availability; a copy in Good condition may currently be on
loan.
```

8. Demonstrate Return UPDATE Safely

```
START TRANSACTION;
UPDATE Loans SET ReturnedOn = '2026-10-22' WHERE LoanID = 9015;
SELECT LoanID, CopyID, ReturnedOn, OpenCopyID FROM Loans WHERE LoanID = 9015;
ROLLBACK;

Expected output during the transaction:
9015 | 506 | 2026-10-22 | NULL

Explanation: setting ReturnedOn releases the copy because OpenCopyID becomes NULL. ROLLBACK restores the
original sample state after the demonstration.
```

9. Conclusion

These examples demonstrate filtering, sorting, grouping, group filtering, all required aggregate functions, and a reversible update transaction.